1. Discuss an efficient method to sort ten "10 digits" numbers in almost

Approach — ~~The~~ ~~opti~~

To sort small fixed no. of large integers (like 10, -digit numbers) in nearly constant time, we utilize Radix Sort.

$a+b$
$a = a+b+ab$
$= (a+1)b + a$

dx-

While the general complexity of Radix Sort is $O(d.(n+k))$ where d is the no. of digits, n is the no. of elements, and k is the range of the base. it approaches $O(1)$ in this ~~sp~~ specific scenario.

Because, n is constant ($n=10$)
d is constant ($d=10$)

The method ① Stable sort ~~of~~ Digit! starting from the largest significant digit to the most significant digit performs a stable sort (usually counting sort) on each position.

② counting sort as the Base! since each digit is 0-9, the bucket size k is always 10.

③ ~~why~~ As in Big O notation if the input size and the ~~output size~~ number of process are fixed the execution time does not scale with a variable n. it performs exactly $10 \times (10+10)$ operations, which is a fixed overhead.

2. Converting knapsack to Satisfiability (SAT)

The 0/1 knapsack problem can be mapped to a SAT problem by treating the inclusion of each item as a boolean variable.

Mapping via State Space Tree:-

variable:- Define boolean variables

$x_i \rightarrow i \rightarrow [1, n]$

where

$x_i = 1$ means item i is included and $x_i = 0$ means excluded

Constraints -

Capacity constraint:

$\sum w_i x_i \leq W$ in SAT

Term this is representation of clauses that forbid combinations of variables whose weight exceed W.

Profit Goal → $\sum p_i x_i \geq P$ clauses are constrained to ensure the sum of profit meets a required threshold.

State Space tree Connections

Each path from the root to a leaf in the state space tree represents a unique assignment of truth values to $(x_1 \ldots x_n)$.

if a path satisfies the capacity and profit constraints, that assignment makes the boolean formula satisfiable.

3) LPS or pi table →

The pi table is used in KMP Algo. to skip necessary comparisons

String	pi table
aaaaabbaa	[0, 1, 2, 3, 4, 5, 0, 0, 1, 2]
aaaa	[0, 1, 2, 3]
abcabcabc	[0, 0, 0, 1, 2, 3, 4, 5, 6]

4) Algorithm Complexity Examples →

Excluding Standard Searching (linear/Binary) & Sorting (Bubble/quick/merge) →

O(1) → Constant time.

↳ Accessing an element in an Array : Retrieving a value via an index (e.g. arr[5]).

Push/Pop in a Stack : Adding or removing from the top of a linked-list based stack.

O(n) - Linear time

· Finding the minimum/max. → Iterating through an unsorted list once to find the largest value.

Strin removal

O(n²) → Matrix Addition. (n×n) → O(n²)

O(n³) → Matrix multiplication (Naive Approach)

O(log n) → Balanced BST operations.

O(n log n) → Fast Fourier Transformation.